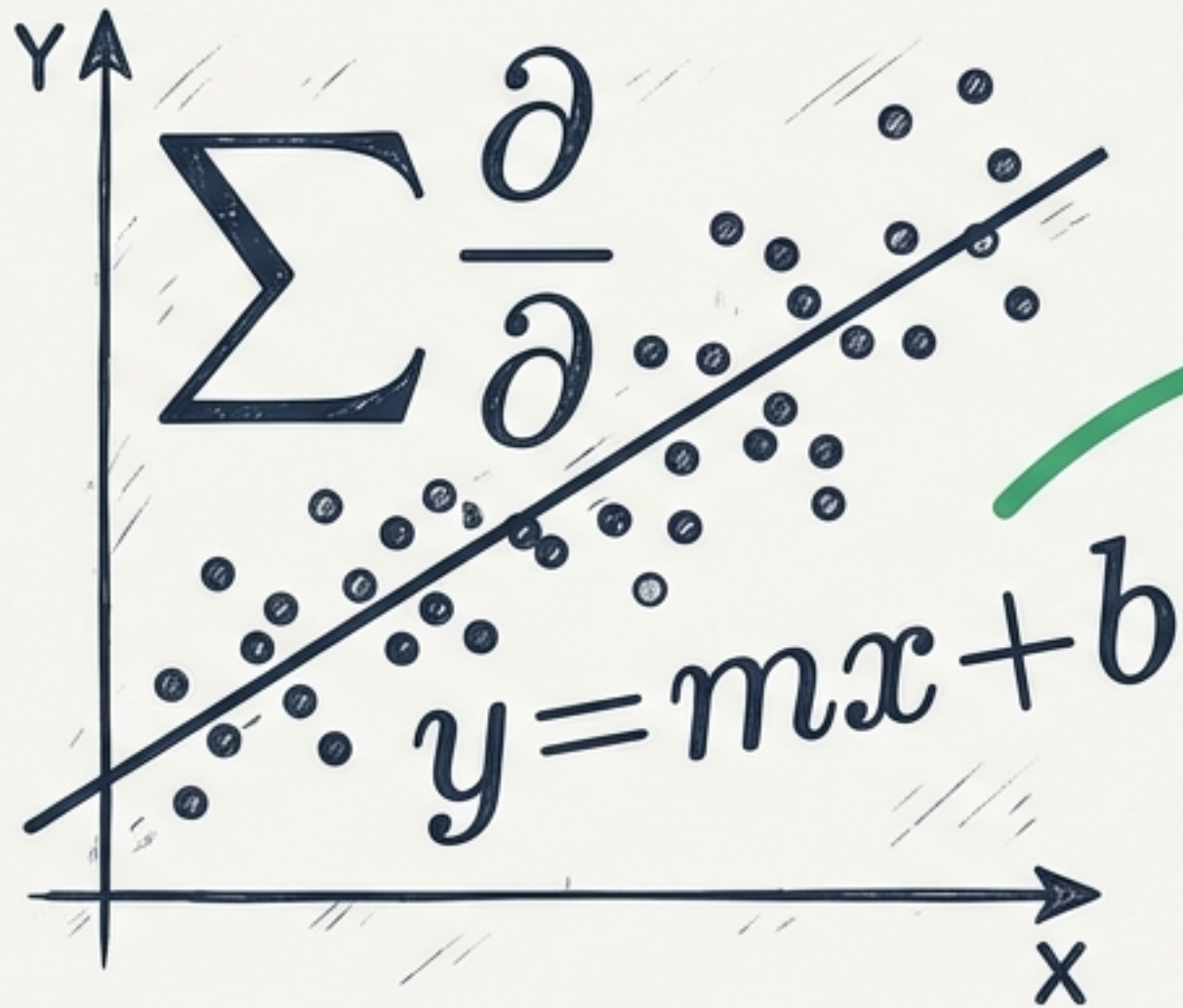


Simple Linear Regression: From Math to Machine

A First-Principles Derivation and Python Implementation from Scratch.

The Math World

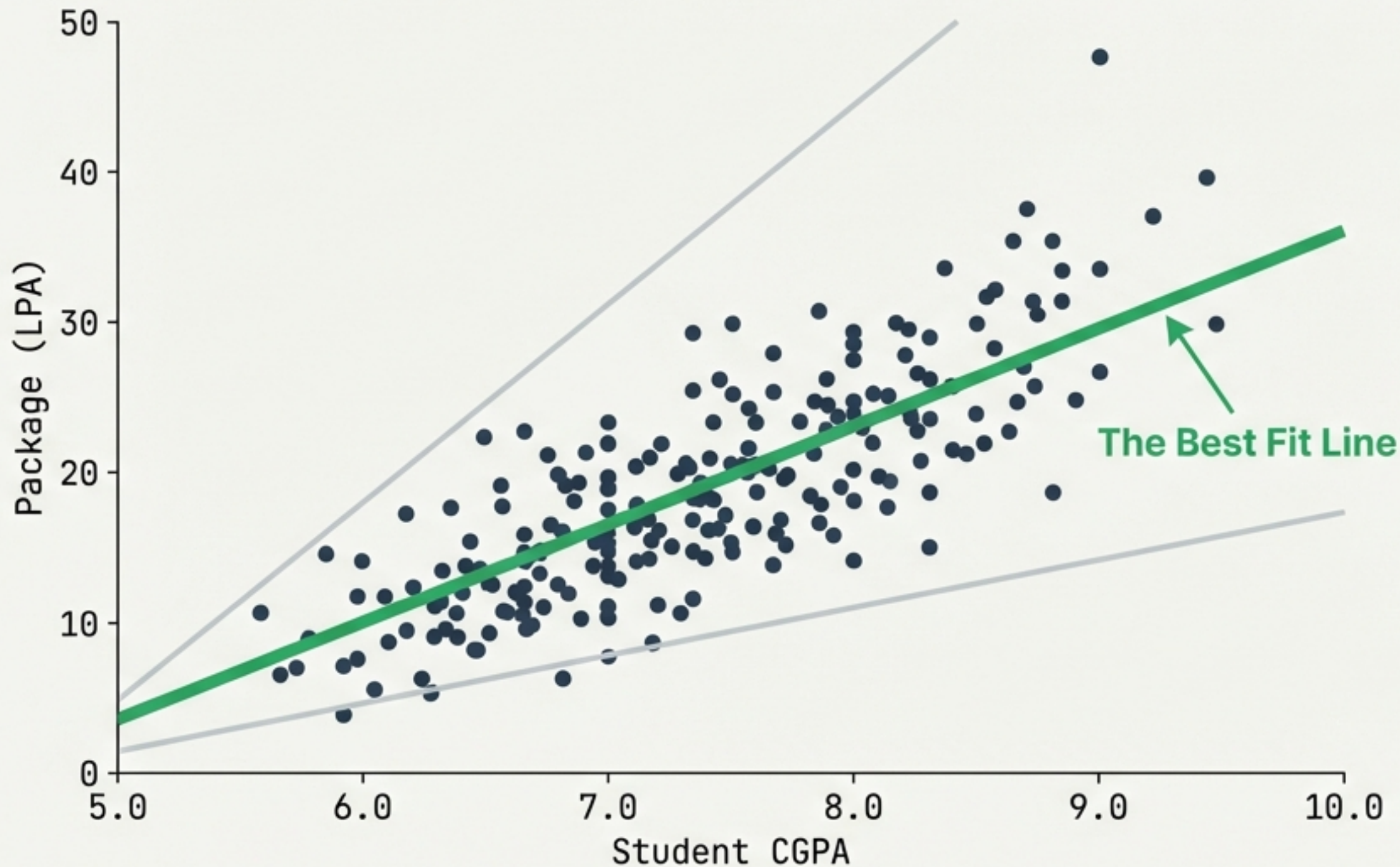


The Code World

```
class MeraLR:  
    def fit(self, X, y):  
        # Implementation...
```

Based on the derivation of Ordinary Least Squares (OLS).

The Core Problem: Finding the 'Best Fit'



The Context: We have a dataset of students (CGPA) vs. placement packages. The relationship is linear.

The Goal: Find a model $y = mx + b$ that predicts package values.

The Conflict: No line touches all points. Every line makes mistakes.

The Insight: The best line is the one that makes the *minimum* total mistakes.

Two Roads to the Optimal Slope

Calculate
m and b

Closed Form Solution (OLS)

- Direct Mathematical Formula
- Exact Solution
- Standard for Simple Linear Regression

****Our Focus Today****

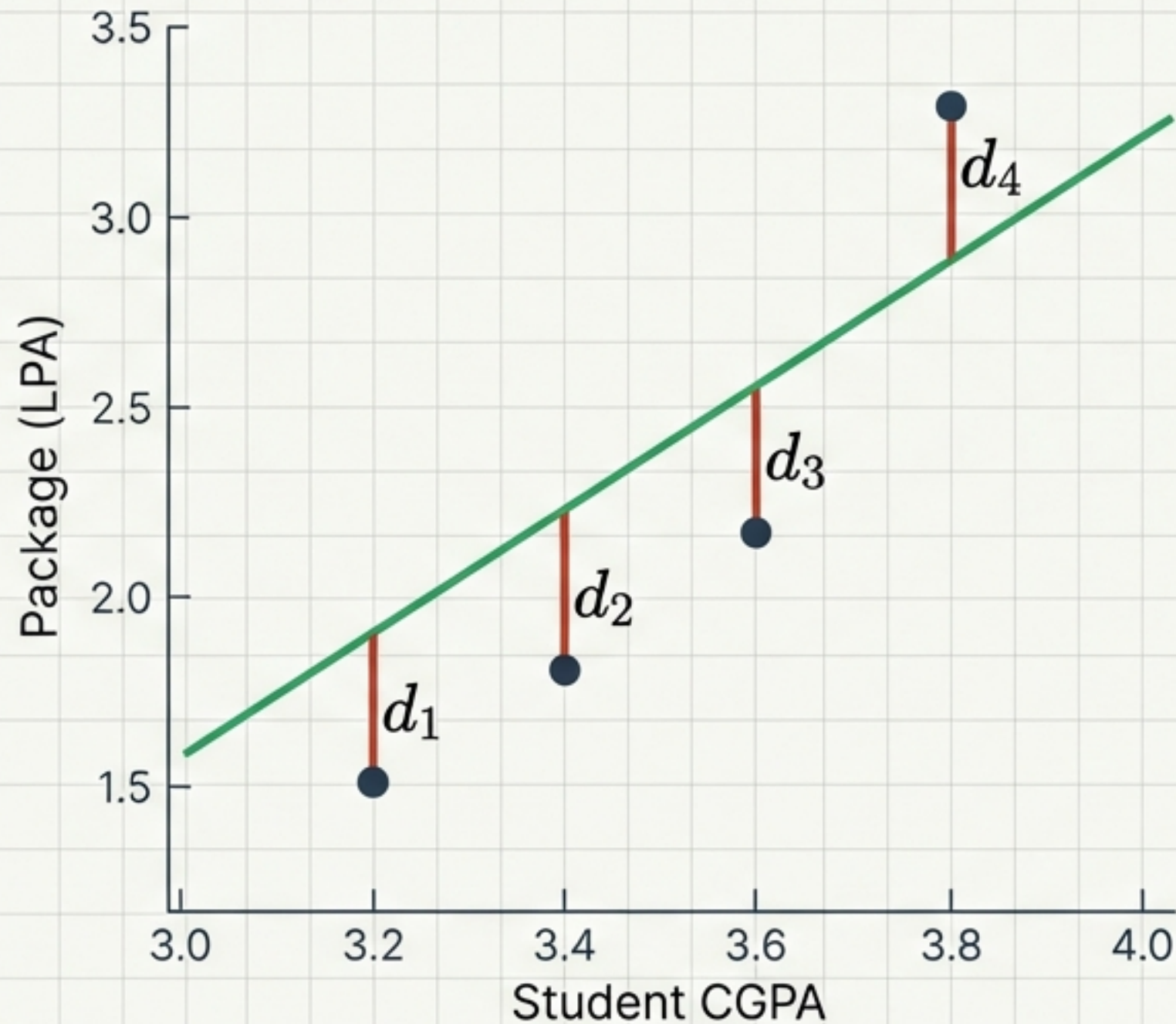
Non-Closed Form (Gradient Descent)

- Iterative Approximation
- Essential for High Dimensions
- Computationally intensive

The Strategy: We will use Ordinary Least Squares (OLS).

For 1D data, OLS provides an instant, exact formula without the need for iterative guessing.

Quantifying Failure: The Error Function (E)



$$E = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$E = \sum_{i=1}^n (y_i - (mx_i + b))^2$$

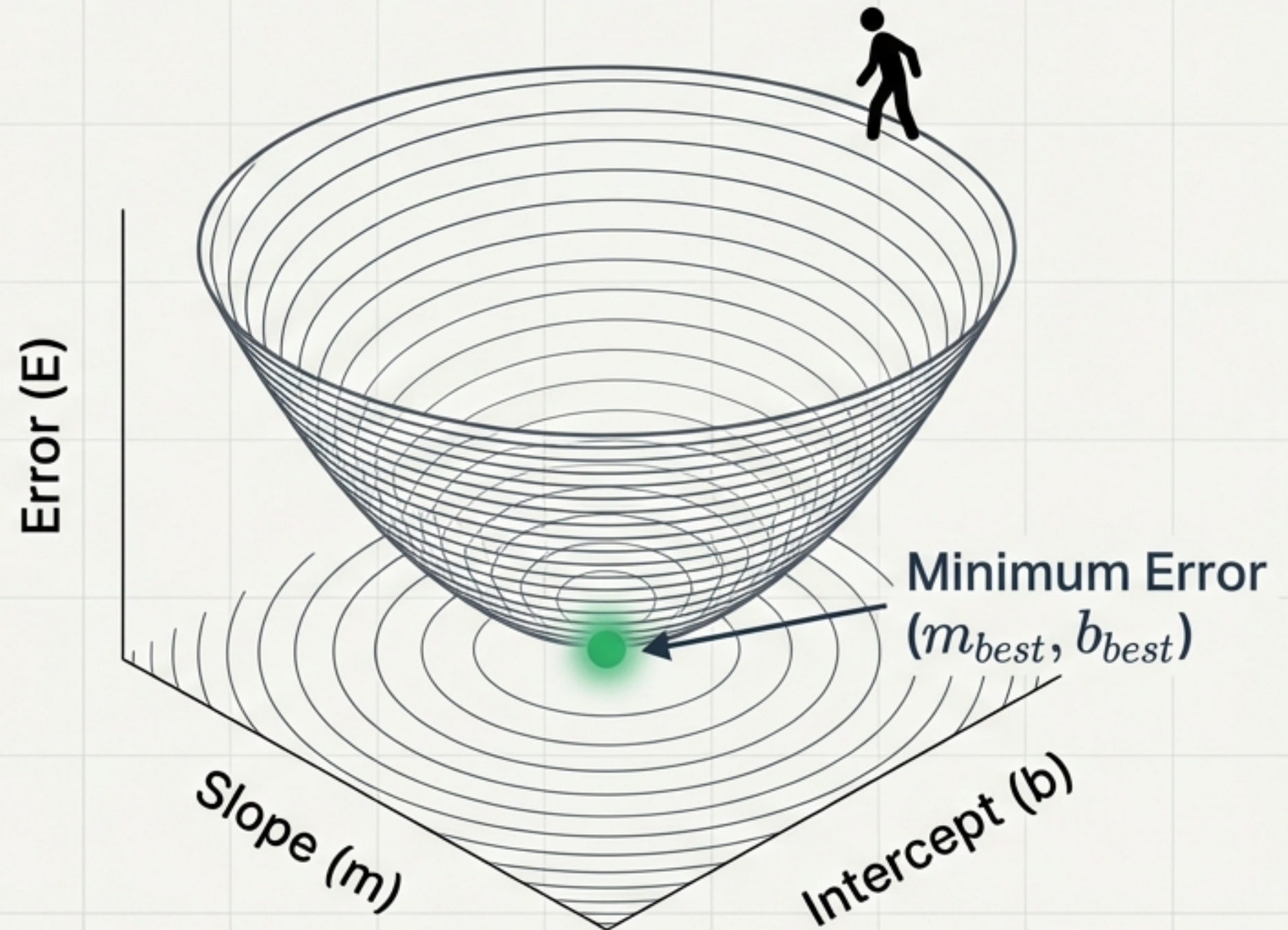
****Why Square the Errors?****

1. ****Negatives:** Squaring eliminates negative differences, preventing errors from cancelling each other out.
2. ****Differentiation**** The squared curve is a smooth parabola, making it differentiable at all points (unlike absolute value).

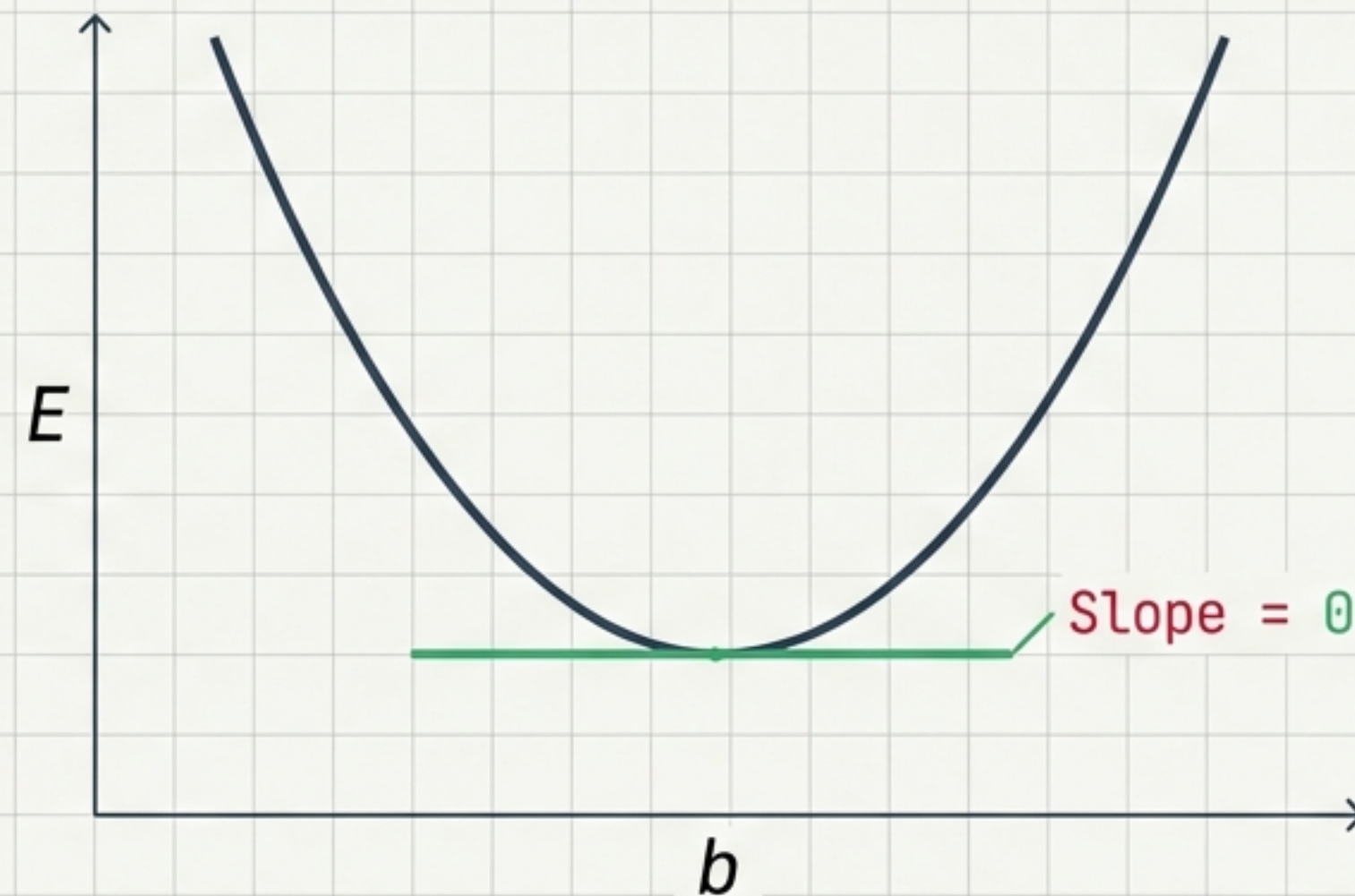
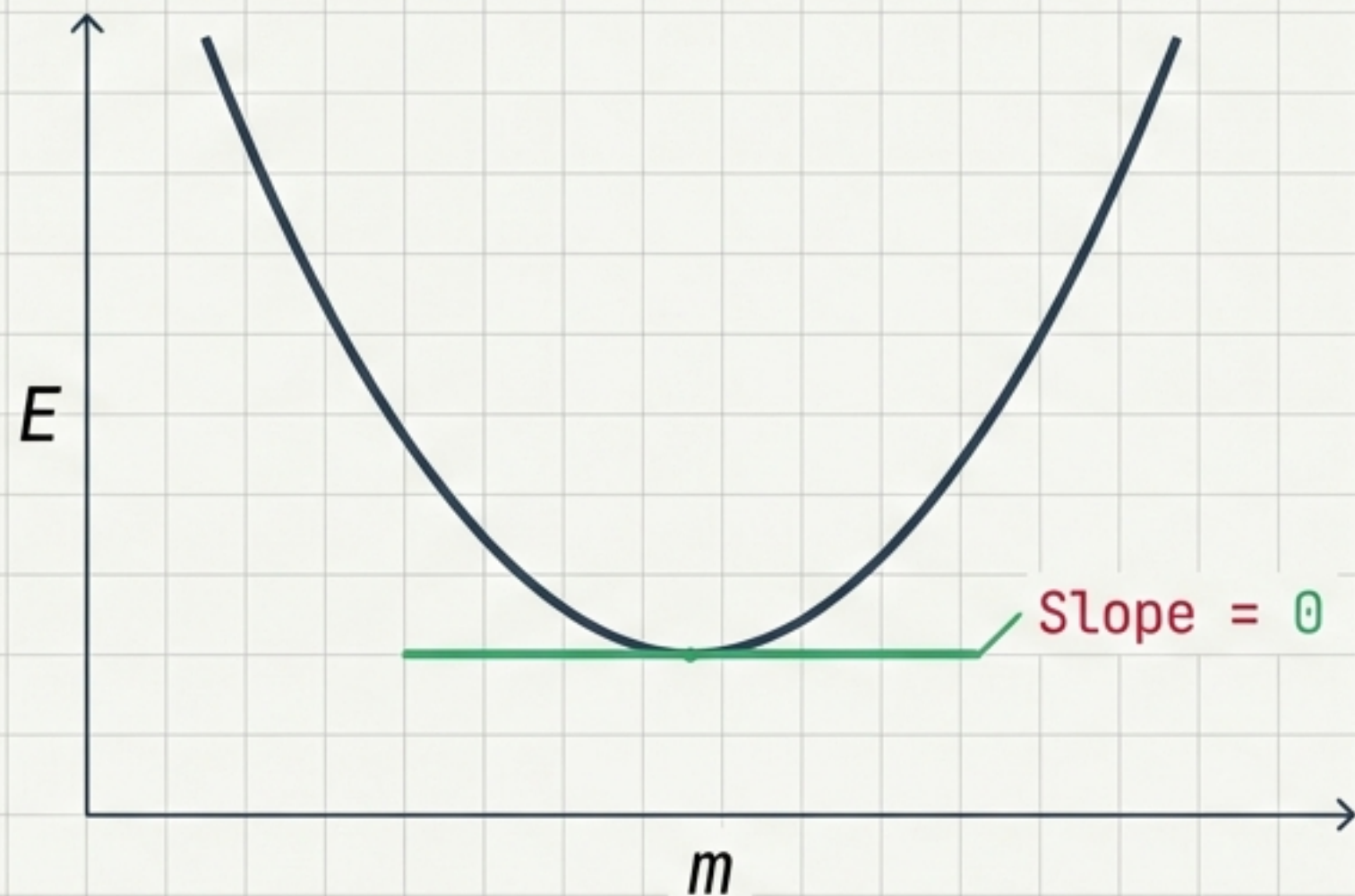
The Geometry of Loss

The Landscape: Since the data (x, y) is fixed, the Error (E) changes only when we adjust our line parameters m and b .

The Mission: We need to find the coordinates at the very bottom of this valley.



The Calculus Shortcut: Partial Derivatives



The Rule: At the minimum point of a smooth curve, the derivative (slope) is exactly zero.

The Action Plan:

1. Differentiate E with respect to b ($\frac{\partial E}{\partial b}$) and set to 0.
2. Differentiate E with respect to m ($\frac{\partial E}{\partial m}$) and set to 0.
3. Solve the system of linear equations.

Step 1: Deriving the Intercept (b)

Step 1: The Derivative

$$\frac{\partial E}{\partial b} = -2 \sum_{i=1}^n (y_i - mx_i - b) = 0$$

Step 2: Simplify and Expand

$$\sum y_i - m \sum x_i - \sum b = 0$$

Step 3: The Mean Trick

(Annotation: Since $\sum b = n \cdot b$)

$$n \cdot b = \sum y_i - m \sum x_i$$

$$b = \frac{\sum y_i}{n} - m \frac{\sum x_i}{n}$$

$$b = \bar{y} - m\bar{x}$$

Interpretation: The intercept adjusts based on the means ($\bar{y}$, $\bar{x}$) and the slope (m).

Step 2: Deriving the Slope (m)

Step 1: Substitute b

Substitute $(\bar{y} - m\bar{x})$ into the error equation $E = \sum (y_i - mx_i - b)^2$ and differentiate with respect to m (setting to 0).

Step 2: Isolate m

$$\sum_{i=1}^n (y_i - \bar{y}) = m \sum_{i=1}^n (x_i - \bar{x})$$

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Covariance: How x and y move together.

Variance of x : How spread out x is.

The Closed Form Solution

1. The Slope (m)

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

2. The Intercept (b)

$$b = \bar{y} - m\bar{x}$$

We have successfully 'invented' the algorithm. We can now calculate the perfect line for any 2D dataset instantly. Now, let's build the machine.

Architecture: Building the MeraLR Class

MeraLR

`m : float (initially None)`

`b : float (initially None)`

`__init__()` : Initialize parameters

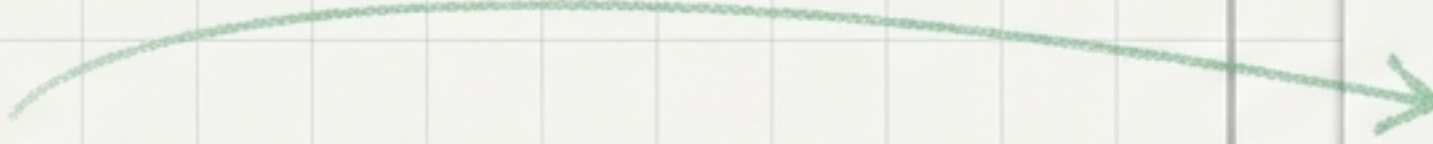
`fit(X_train, y_train)` : Calculate formulas, update m & b

`predict(X_test)` : Return $Y = mX + b$

Objective: Build a Python class from scratch that replicates the behavior of `sklearn.linear_model.LinearRegression`.

Coding the Logic: The 'fit' Method

The Math

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$


The Code

```
def fit(self, X, y):  
    num = 0  
    den = 0  
    for i in range(X.shape[0]):  
        # The Summation Loop  
        num = num + ((X[i] - X.mean()) * (y[i] - y.mean()))  
        den = den + ((X[i] - X.mean())**2)  
  
    self.m = num / den  
    self.b = y.mean() - (self.m * X.mean())
```

Coding the Logic: The 'predict' Method

Input
(X_test)



Model
(Apply $y = mx + b$)



Output
(Predicted Package)

```
def predict(self, X_test):  
    return self.m * X_test + self.b
```

Simple linear algebra



Once trained, prediction is simple linear algebra. The method takes an input array (CGPA) and broadcasts the equation across it to return predicted Packages.

Execution: Training the Model

```
In [1]: import pandas as pd
        from sklearn.model_selection import train_test_split

        # 1. Load and Split Data
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

        # 2. Train Custom Model
        lr = MeraLR()
        lr.fit(X_train, y_train)
```

```
In [2]: print(lr.m)
        print(lr.b)
```

```
Out[2]: 0.55
        -0.81
```

The Verdict: MeraLR vs. Scikit-Learn

MeraLR (Our Code)

Slope (m): **0.55**

JetBrains Mono

Intercept (b): **-0.81**

JetBrains Mono

Scikit-Learn (Standard)

Slope (m): **0.55**

JetBrains Mono

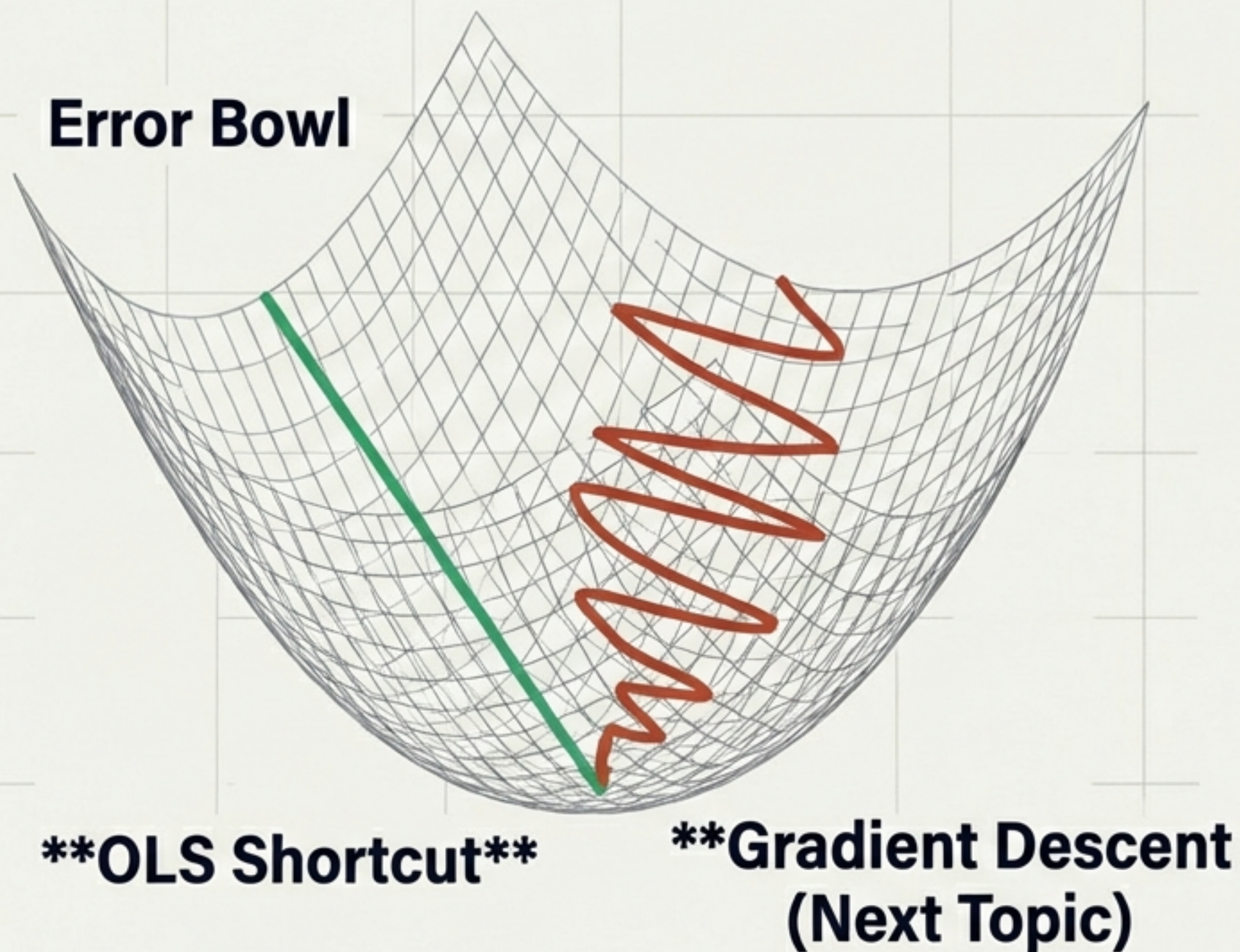
Intercept (b): **-0.81**

JetBrains Mono



Conclusion: Exact Match. We have successfully reverse-engineered the logic inside the black box. The standard `LinearRegression` class uses the exact OLS formulation we derived.

The White Box Revealed



- **1. First Principles:** We defined 'Best Fit' as minimizing squared errors.
- **2. Calculus:** Derivatives gave us the 'Closed Form' solution.
- **3. Code:** We translated the math directly into a Python class.

While OLS is perfect for 1D data, complex high-dimensional problems require the iterative approach (Gradient Descent).